Teorie:

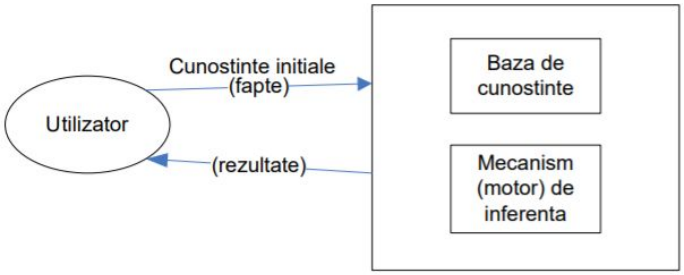
1. Structura sistemelor bazate pe cunostinte
2. Tip de similitudine
3. Reguli de formare a unui limbaj, formule
4. Inferenta logica, lant inferential
5. Cautarea optimala x3
6. CF (definitie + comentarii)
7. Parcurgerea in adancime pentru arbori/grafuri - algoritm
8. Etapele sistemului rezolutiv
9. Functia de apartenenta si logica Fuzzy
10. Inferenta Fuzzy
11. Particularitatea reprezantarii prin cadre
12. Reprezentarea cunostintelor prin reguli de productie
13. Reprezentarea prin limbaje de ordin I
14. Cautarea in largime
15. Incertitudini
16. Comis voiajor
17. Castig informational
18. Inconsistenta
19. Consistenta BC(…. Booleana)
20. Defazificare(acoperire + centru masa)

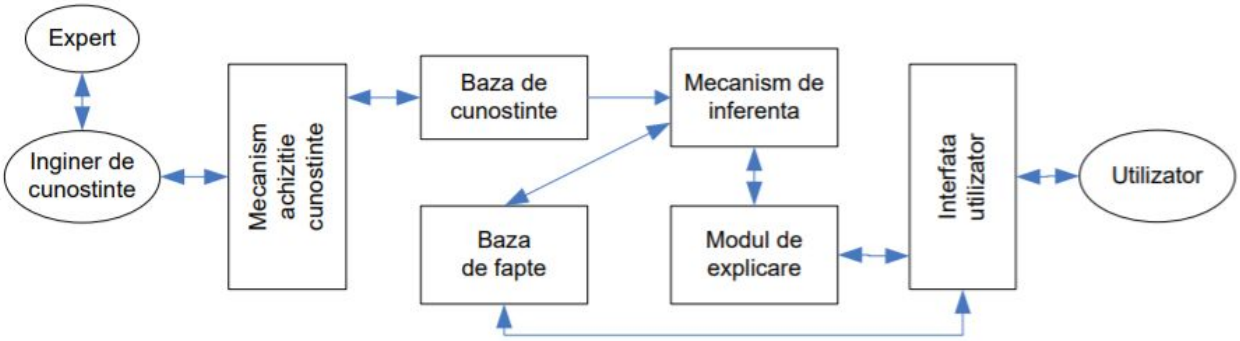
Probleme:

1. Prolog: Numarul de aparitii al lui x din doua liste
2. Scrieti un program in Prolog care calculeaza suma elementelor unei liste
3. Problema cu castig informational
4. Problema cu CF
5. Sa se obtina din doua liste, o lista ce contine elementele care se gasesc intr-o lista si nu se gasesc in cealalta.
6. Sa se obtina din doua liste, o lista ce contine elementele comune celor doua liste.(intersectia)
7. Def reguli + transpunere prolog
8. Def piese de …. Fuzzy -> rezultatul inferentelor

Rosu – exams + subiecte spuse de prof
Albastru – subiecte spuse de prof

1. Structura generala si componentele sistemelor bazate pe cunostinte





Baza de cunostinte – pastreaza cunostintele din domeniul de expertiza

Ce poate contine BC?

-Constante cu diverse tipuri de date (temperatura absoluta,numele unei personae etc)

-Variabile – reprezentate de date achizitionate prin diverse mijloace

- Diverse relatii intre obiecte (distanta dintre doua localitati, relatii de rudenie etc)

-Reguli specifice domeniului

Inginerul de cunostinte extrage prin diverse tehnici cunostintele de la expert si le formalizeaza adecvat limbajului de reprezentare utilizat.

Mecanismul de achizitie cunostinte este un utilitar care ajuta inginerul de cunostinte la crearea si gestionarea bazei de cunostinte (adaugare/stergere cunostinte, modificare

cunostinte, asignare ordine).

Baza de cunostinte – stocata in memorie permanenta compusa din piese de cunoastere (fapte, reguli, metacunostinte).

Baza de fapte – stocata in memorie temporara, pastreaza faptele initiale introduse de utilizator si contextul curent al rezolvarii problemei (alte fapte demonstrate)

Mecanismul de inferenta

– implementeaza elementele de control pentru rularea programului, implementat ca program procedural.

-Contine un algoritm de control generic, acelasi indiferent de continutul bazei de cunostinte

-Determina momentul in care o piesa de cunoastere este utilizata

-Secventa de operatii intre inceperea si terminarea programului nu este dictata de programator, ci determinate de cunostintele initiale

Modul de explicare – utilitar care furnizeaza utilizatorului piesele de cunoastere utilizate, de ce are nevoie de fapte la anumite momente ale executiei, justifica rationamentul efectuat.

Interfata utilizator – faciliteaza interactiunea dintre utilizator si system.

Sistemul esential este compus din:baza de cunostinte (BC), baza de fapte (BF), mecanism de inferenta (MI).

Aceste componente sunt obligatorii la orice implementare, celelalte sunt module auxiliare si pot lipsi, functie de implementare si specific system.

Baza de cunostinte se modifica la intervale mari de timp (cunostintele evolueaza lent).

2. Tip similitudine

Tip de similitudine (abstractizare fundamentala pe care sunt construite limbajele).

$\Sigma = < I, J, K, \theta, \varphi >$ in care:

I – multimea de indexarea a familiilor de relatii. J – multimea de indexare a familiilor de functii
K – multimea de indexare pentru elemente distincte
θ:I -> N aplicatia de aritate a familiilor de relatii, cu N multumea nr natural

φ : J -> N aplicatia de aritate a familiilor de functii, cu N multimea nr natural

Structura L_α avand tipul de similitudine α este un cvadruplu cu $i \in I$, $j \in J$, $k \in K$ format din:

- Multimea A definite si suportul sau universal
- Familia $\{R_i^\alpha \mid i \in I\}$ astfel incat pentru fiecare $i \in I$, R_i^α este o relatie pe A cu aritatea $\theta(i)$
- Familia $\{ F_j^\alpha \mid j \in J\}$ astfel incat pentru fiecare $j \in J$, F_j^α este o functie pe A cu aritatea $\varphi(j)$
- O submultime $\{ e_k^\alpha \mid k \in K\}$ a lui A, denumita si multimea elementelor distincte din A
- Daca I = Ø, adica stuctura nu contine simboluri relationale se numeste algebra
- Daca J =Ø, adica structura nu contine simboluri functionale, structura se numeste system relational

- Un limbaj de ordinal intai e constituit din simboluri primitive, termini, formule atomice, formule

Exemplu: Structura α = < N, ≤, +, *, s, 0 > o aritmetica in N:

- ≤ simbol relational binar
- +, * simboluri functionale binare
- S, simbol functional successor s(x) = x + 1
- 0, constanta 0

3. Reguli de formare limbaj

In formarea oricarui limbaj se porneste de la simboluri primitive -> termeni -> formule atomice -> formule.

Termeni : un simbol primitiv poate fi un termen. Daca t1,t2,...tn, aplicarea oricariu simbol functional asupra lor produce un termen trebuie sa contina o sematica.

Aplicarea repetata a regulilor produce noi termeni. Un termen poate fi o formula atomica.

Daca t1,t2,...tn – termeni si r simbol rationat => r(t1,t2,...tn) formeaza o formula atomica.
Formula : daca F1, F2 sunt formule atomice, atunci utilizare colectivelor logice determina noi formule. Orice formula atomica are asociata valoare metalingvistica de A/F.

Valori de adevar formule

•In semantica oricarui limbaj, oricarei formule i se asociaza o semnificatie si o valoare de adevar

•O expresie este valida daca valoarea sa de adevar este totdeauna adevarat indiferent de valorile de adevar ale componentelor sale

•O expresie este consistenta daca valoarea sa de adevar nu este intotdeauna falsa

•O expresie este invalida daca valoarea sa de adevar nu este intotdeauna adevarat

•Daca valoarea de adevar a unei expresii este totdeauna fals ea se numeste invalida

•Expresiile care au valoare adevarat sau fals functie de valorile de adevar ale formulelor atomice se numesc contingente

4. Inferente, lant inferential

Regulile de inferenta sunt metode prin care se determina consecintele unor presupuneri despre formule sau enunturi

Inferenta – operatia prin care pe baza acceptarii unui numar finit de enunturi numite premise se trece la acceptarea unui alt enunt numit concluzie. Premisele formeaza conditia suficienta a concluziei chiar daca concluzia se poate obtine si din alte premise

Prin inferenta valoarea de adevar a premisei se transfera concluziei

Un sir de inferente in care premisele unora se gasesc printe concluziile altora se numeste **lant inferential**

Clauza Horn/Teoria inferentelor logice

Daca A1 /\ A2 /\ ... /\ An atunci B

Specifica sistemelor bazate pe reguli

B daca A1 /\ A2 /\ ... /\ An

Premisa si concluzie complexa :

- Daca A1 /\ A2 /\ ... /\ An atunci B1 \/ B2 \/....\/Bm
- B1 \/ B2 \/....\/Bm daca A1 /\ A2 /\ ... /\ An

Teoria inferentelor logice

In programele inteligente, sistemul axiomatic si inferential trebuie sa coexiste.

Cea mai uzuala modalitate de reprezentare a regulilor de inferenta foloseste forma clauzala

Premisa complexa consecinta unica:

- Daca A1^A2 ^…. ^An atunci B
- B daca A1^A2 ^…. ^An

Premisa si concluzie complexa:

- Daca A1^A2 ^…. ^An atunci B1˅B2˅….˅Bm
- B1˅B2˅….˅Bm daca A1^A2 ^…. ^An

Forma generala ∀x1,x2,…,xk,B1,B2,…Bm <- A1,A2,…,An in care s-au inlocuit conjunctiva si disjunctive cu virtule iar Xi sunt variabile

Clauza Horn provine dintr-o implicatie de forma A1,A2,..An -> B sau in scriere inversa B daca A1,A2,…An sau b<-A1,A2,…,An

Daca P si Q sunt doua enunturi, atunci:

- Negatia lui Q notata (7Q) afirma ca Q nu are loc
- Conjunctia lui P si Q, notata (P^Q), afirma ca P si Q are loc
- Disjunctia lui P si Q, notata (P˅Q), afirma ca fie P, fie Q are loc
- Implicatia: P implica Q(P=>Q), afirma ca daca P are loc, atunci sigur Q are loc
- Echivalenta lui P si Q(P≡Q) afirma ca P si Q au loc in acelasi timp

- Cuantificatorii pot fi utilizati doar prin atasarea la variabile ale predicatelor si permit formarea de enunturi compuse.

Daca un predicat P cu variabila x afirma ca obiectul instantiate la variabila x are proprietatea prin enuntul ∃(x)P(x), se intelege ca in clasa de obiecte din care face parte x macar un obiect are proprietatea

Similar ∀(x)P(x) afirma ca orice obiect al clasei din care face parte x are proprietatea

O informatie are valoare de adevar ce se inscrie in diferite scale. Logica binara opereaza cu valorile extreme adevarat/fals, logica fuzzy e multivalenta reprezentata de o functie continua.

Reguli de formare formule

Pentru formarea formulelor se dau regulile:

- Daca a si b sunt propozitii, atunci a, b, 7a, 7b, aˇb, aˆb, a=>b, a≡b sunt formule atomice ale calculului propozitional
- Orice formula atomica e o formula
- Daca F1 si F2 sunt formule, 7F1, 7F2, F1ˇF2, F2 F1=>F2,F1≡F2 sunt formule
- Orice formula inclusa intre paranteze este o formula

Tipuri de formule

– **Tautologii** – formule ce au tot timpul valoarea adevarat indiferent de valoarea de adevar a propozitiilor componenter

– **Contradictii** – formule ce au totdeauna valoarea fals indiferent de valoarea de adevar a propoizitiilor componente

– **Formule variabile** – formule ale caror valori de adevar sunt dependente de valoarea de adevar a propozitiilor componente.

Caracteristici model reprezentare:

- Expresivitate – capacitatea de a descrie obiectele si propriet lor esentiale
- Claritate – relatiile dintre obiecte sunt clar explicitate
- Completitudine – repreznetare complete si concise fara detalii nesemnificative
- Independent de context si neambiguu semantic, nu trebuie sa fie dependent de context cum se intampla usual in limbajele natural
- Eficienta – din punctul de vedere al gestionarii bazei si al executiei rationamentelor

Reprezentarea in limbaje de ordinul intai

Este derivata din logica matematica prin care pisele de cunoastere sunt descrise de formule in calculul cu predicate cu sintaxa si semantica din logica matematica.

Avantaje:

- Regulile de inferenta sunt regulile de deductive din calculul cu predicate
- Rationamente computationale clare(demonstrarea automata a teoremelor)
- Faptele deduse sunt garantate

Dezavantaj: dificultatea de gasire a trasaturilor relevante ale pieselor de cunoastere

Etape reprezentare calcul cu predicate:

- Captarea piesei de cunoastere exprimata in limbaj natural
- Descompunerea acesteia in propozitii
- Legarea propozitiilor prin connevtive logice
- Exprimarea fiecarei asertiuni printr-un predicat cu nr de locuri finit in care sunt incluse variabile individuale sau constant

Reprezentarea prin reguli de productie

Regulile de productie isi au originea in sistemele care pornesc de la un context initial caruia I se aplica o serie de transformari successive.

Partea de conditie mai e numita si parte contextuala si indica contextual in care regula e aplicabila, iar partea de actiune se numeste si parte transformationala pentru ca aceasta determina modificarea contextului

Sintactic este asemanatoare cu instructiunea IF…THEN din programarea clasica.

Ordinea faptelor si a pieselor de cunoastere nu are nici o importanta, ele sunt declansate de mecanismul de control al motorului de inferenta.

Structura regula productie

Efectuarea inferentelor e facuta de un mechanism independent cu grad mare de generalitate.

Fiecare regula e modulara si independenta, dar la cresterea complexitatii cunostintele nu mai sunt total independente si trebuiesc urmarite corelatiile dintre ele, pt a asigura consistenta bazei de cunostinte.

Interpretarea sistemelor de productii

- Corespondenta – selectare reguli ce satisfac partea de conditie
- Rezolvarea conflictelor – alegerea din multimea regulilor ce satisfac partea de conditie a unei reguli ce va fi aplicata
- Executie
- Actualizare context
- Oprirea mecanismului interpretativ

Un system de productii e definit ca SP = <K, P, σ, ω, ps>.

Inconsistenta sistemelor de productii

Un sistem de productii este inconsistent daca pornind de la o baza de fapte initiala se poate obtine validarea si invalidarea unei expresii.

Exemplu:

• BF (constructie=rezistenta, cutie=aspectoasa, conectare=puternica)

• Sistemul de productii:

P1: IF (constructie=rezistenta AND cutie=aspectoasa)

THEN (confectie=aluminiu)

P2: IF (cutie=aspectoasa) THEN (surub = cap ingropat)

P3: IF (conectare=puternica)THEN (surub= NOT(cap ingropat))

Se observa ca baza de fapte initiala (constructie=rezistenta, cutie=aspectoasa, conectare=puternica) este consistenta

Aplicarea regulilor produce prin derivare urmatoarea baza de fapte (constructie=rezistenta, cutie=aspectoasa, conectare=puternica, confectie=aluminiu, surub=cap ingropat, surub=NOT(cap ingropat))

Am obtinut concluzii contadictorii adica adevarat si fals pentru aceeasi expresie.

Retele semantice

• O retea semantica este un graf orientat etichetat

• Etichetele nodurilor: concepte, instante, valori, proceduri

• Arcele sunt etichetate cu relatiile dintre noduri. Spre exemplu un nod concept leaga un nod instanta printr-un arc este

o (este un) – relatie de apartenenta, un concept generic indica un concept prin arcul etichetat subset – relatie de

incluziune.

• Nodurile de nivel superior sunt asociate conceptelor generice, iar cele de baza sunt instante

• Arcele se mai numesc si primitive taxonomice intrucat corespund unei clasificari

Reprezentarea prin cadre

Cadru - structura identificata printr-o eticheta (nume cadru)

Cadrele reprezinta colectii de obiecte ce au atasate caracteristici si intre obiecte avem legaturi.

Structura unui cadru nu este rigida, poate fi modificata prin interventia demonilor asociati cadrului.

Contine perechi caracteristica (atribut, proprietate) – valoare (slot value). O caracteristica se refera la o relatie de apartenenta/incluziune, dar si la alte proprietati

Tipuri demoni:

- Default : valoare implicita

- If needed : nu poate fi data ca valoare implicita, este caruta

- If added : activata la introducerea valorii caracteristii la care este atasata

- If removed : activata la indepartarea valorii caracteristicii.

Scenarii

•Utilizate pentru a reprezenta evenimente. Este un caz particular de cadre

•Un scenariu este o structura ce descrie o secventa de evenimente

•Numele scenariului identifica structura ce contine perechi caracteristica valoare

•Se prezinta un exemplu de scenariu “desfasurare prezentare lucrare stiintifica”

8. Caracterizare sisteme rezolutive

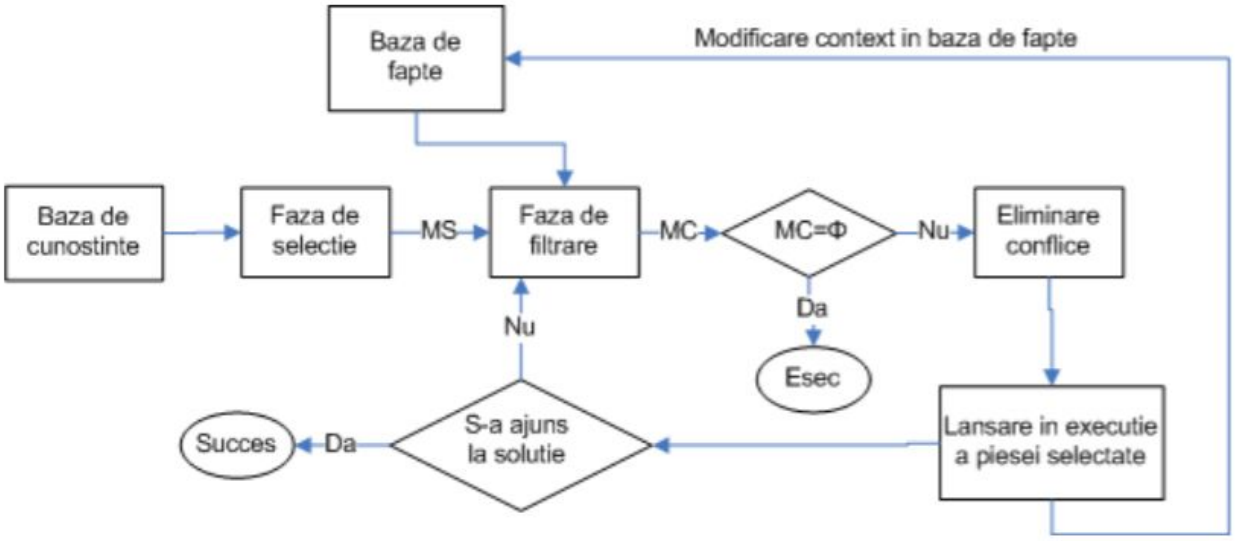
Sistemul rezolutiv este acea parte a unui SBC care utilizeaza piesele de cunoastere pentru realizarea rationamentelor

Implementeaza un mecanism generic de control al utilizarii pieselor de cunoastere

Cautarea unei solutii presupune construirea si parcurgerea de arbori sau grafuri

Arborii/grafurile nu sunt cunoscute aprioric ci construite dinamic la utilizarea pieselor de cunoastere

Un sistem rezolutiv implementeaza o strategie de control in scopul gasirii unei solutii.



Fazele sistemului rezolutiv

Faza de selectie – Se extrage din baza de cunostinte piesele de cunoastere specifice formularii problemei. Piesele extrase formeaza multimea selectata (MS). Are sens numai daca baza contine metacunostinte.

Faza de filtrare – Din MS se extrag piesele de cunoastere ce au premisa adevarata, adica expresiile din premisa se gasesc in baza de fapte . Se obtine multimea candidata (MC) care se mai numeste si multime conflictuala intrucat contine candidatii la executie.

Faza de eliminare conflicte – determina din multimea candidata piesa de cunoastere ce va fi lansata in executie.

Faza de lansare in executie – piesa selectata este lansata in executie si concluziile sale sunt adaugate in baza de fapte, modificand contextul curent al problemei.

Strategii de control inainte

Strategia de control inainte porneste de la ipoteza si aplica succesiv piesele de cunoastere aplicabile pana la gasirea solutiei sau pana cand multimea candidata devine vida. Tranzitia de la un context la altul este rezultatul aplicatii unei piese de cunoastere. Starea finala poate sau nu sa satisfaca obiectivul. Este o strategie in care favorizam piesele de cunoastere cu mai multe formule in premisa.

Strategii de control inapoi

In acest caz problema formulata este descompusa in subprobleme pornind de la scop la obiectiv. Problema este rezolvata atunci cand toata subproblemele sunt demostrate. Se porneste de la context de stare finala -> Aplicare piesa de cunoastere (PC) -> Context de stare precedenta …. Solutia reprezinta un graf in care toate nodurile frunza sunt rezolvate. Un graf in care radacina este un nod de tip SI este o solutie daca toti fiii nodului sunt solved. Un graf in care radacina este un nod de tip SAU este o solutie daca cep putin unul dintre fii este solved.

Cautarea solutiei in arbori/grafuri obisnuiti

• Metodele sunt specifice strategiilor de control inainte

• In IA arborii sunt reprezentati cu radacina in sus

• Metodele de cautare se impart in:

– Metode neinformale, dar sistematice – metode ce nu iau in consideratie costul aplicarii operatorilor de tranzitie intre noduri

- • Cautare in largime
- • Cautarea in adancime

– Metode informale

- • Cautare optimala
- • Cautare pe baza apropierii se solutie (gradient)
- • Algoritm A, A*

Notatii:

•Open_list – lista nodurilor atinse dar neevaluate din punctul de vedere al satisfacerii obiectivului •Closed_list – lista nodurilor evaluate

•Informatiile din cele doua liste asigura gasirea unui drum intre starea initiala si starea obiectiv, daca acesta exista

•Structura de date aferenta unui nod contine intotdeauna un pointer catre parintele sau

•Un nod este nod frunza daca nu are nici un nod fiu

Cautarea in largime

Pseudocod algoritm cautare largime:

Initializare stare initiala, stare obiectiv

Open_list={nod_start}

Closed_list=Ø

while (open_list≠Ø) {

n=prim_nod(open_list)

remove(n, open_list)

add(n, closed_list)

if(obiectiv(n)) exit(succes) //testeza potrivire obiectiv

expand(n) // genereaza toate nodurile fiu ni

append(ni,open_list) }

exit(esec)

Cautarea in adancime

Pseudocod algoritm cautare in adancime:

Initializare stare initiala, stare obiectiv

Open_list={nod_start}

Closed_list=Ø

while (open_list≠Ø) {

n=prim_nod(open_list)

remove(n, open_list)

add(n, closed_list)

if(obiectiv(n)) exit(succes) //testeza potrivire obiectiv

```
        expand(n) // genereaza toate nodurile fiu ni
        add_prim(ni,open_list) }
exit(esec)
```

Cautarea optimala

```
Initializare stare initiala, stare obiectiv
Open_list={nod_start(0)}
Closed_list=Ø
while (open_list≠Ø) {
        n=prim_nod(open_list)
        remove(n, open_list)
        add(n, closed_list)
        if(obiectiv(n))
        exit(succes) //testeza potrivire obiectiv
        expand(n) // genereaza toate nodurile fiu ni
        calculeaza g(ni) // g(ni)=g(n)+c(n,ni)
        add(ni,open_list)
        ordonare(open_list,crescator)}
exit(esec)
```

•Daca in arbore sau graf exista un drum de la nodul de start la un nod care indeplineste conditiile de obiectiv algoritmul de cautare optimala gaseste un astfel de drum

•Daca in arbore sau graf exista mai multe drumuri intre nodul de start si noduri care indeplinesc conditia de obiectiv, algoritmul de cautare optimala gaseste drumul optim (cel mai mic cost sau cea mai ridicata eficienta)

•In grafuri un nod se poate gasi in open_list inainte de a fi gasit cel mai bun drum pana la el. Atunci cand se gaseste un drum mai bun se actualizeaza costul si pointerul catre parinte.

Algoritm A

Initializare stare initiala, stare obiectiv

Open_list={nod_start(0)}

Closed_list=Ø

while (open_list≠Ø) {

n=prim_nod(open_list)

remove(n, open_list)

add(n, closed_list)

if(obiectiv(n)) exit(succes) //testeza potrivire obiectiv

expand(n) // genereaza toate nodurile fiu ni

calculeaza f(ni) // f(ni)=g(n)+c(n,ni)-h(n)+h(ni)

if(niЄopen_list and f(ni_new))<f(ni_old)) f(ni)=f(ni_new)

if(niЄclosed_list and f(ni_new)<f(ni_old))

{add(ni,open_list

f(ni)=f(ni_new)

remove (ni,closed_list)}

if(not(niЄopen_list)) add(ni,open_list)

ordonare(open_list,crescator)}

exit(esec)

Logica Fuzzy

• Extinde valoarea de adevar la tot intervalul [0,1]

• O valoare numerica reprezinta posibilitatea ca o declaratie sa fie adevarata sau falsa

• Extinderea teoriei posibilitatii cu un sistem formal al logicii matematice

• Logica fuzzy este concentrata pe cuantificarea si rationarea cu termeni vagi

• Termenii sunt referiti ca variabile lingvistice

• Variabilele lingvistice sunt termeni ai limbajului natural pentru descrierea unor concepte cu valori vagi

SBC fuzzy foloseste variabile lingvistice si piesele de cunoastere sunt denumite cunostinte fuzzy.

In logica booleana un obiect apartine sau nu unui set dat

• Daca ne referim la conceptul copil logica fuzzy da o interpretarea rezonabila privind apartenenta unui persoane la clasa.

• Se observa ca valoarea functiei de apartenenta la set descreste pe masura ce varsta creste devenind 0 la majorat.

Obs: Functiile de apartenenta pot fi definite ca forma standard sau forme liniarizate, numite si singleton.

Inferente Fuzzy

• Un set fuzzy este tratat ca o proprietate fuzzy

• Expresia: x este A are interpretarea ca variabila x apartine setului A

• O regula fuzzy are forma IF x este A Then y este B

• O variabila lingvistica poate sa apartina in acelesi timp la mai multe seturi

Ex: Daca luam variabila Distanta impartita in clasele: zero, foarte_mica, mica, medie, mare clasele alaturate nu sunt disjuncte si ca urmare distanta apartine in acelasi timp la doua clase, insa functia de aparteneta indica cat de mult apartine clasei.

Inferenta fuzzy x este A->y este B

• Se determina valoarea functiei de aparteneta lui x la A

• Se tranziteaza valoarea la apartenetа lui y la B prin una din metodele max-min sau max-produs

Defuzificare

• Suprapunand pe acelasi grafic concluziile celor doua reguli se obtine imaginea din figura.

• Pentru a determina valoarea vitezei se calculeaza centrul de masa pentru suprafata de culoare albastra • Se observa ca astfel se agrega rezultatele celor doua reguli

• In exemplu am utilizat max – produs

Cautarea solutiei cu matricea de costuri – comis voiajur

Caracteristici:

• Graful este complet conectat

• Nodul de start este identic cu nodul obiectiv

• Nu este permisa revenirea la un nod deja atins

• Traseul gasit pleaca din nodul de start, atinge toate nodurile grafului si se incheie in nodul de start

• Legaturile dintre noduri sunt muchii nu arce, pot fi parcurse in orice directie.

Formularea problemei

Componentele unei probleme sunt :

- I – ipoteza

- P – proces

- C – concluzie

Presupunem ca una din componente este necunoscuta (X) :

- IPX care este concluzia aplicarii procedului P asupra ipotezei I. Problema tipica de prelucrare de date (programare clasica)

- XPC care este ipoteza la care aplicand procesul P se obtine concluzia C. Problema rezolvabila daca P este bijectiv, adica X=P-1(C).

- IPX descoperiti procesul aplicat ipotezei pentru a obtine concluzia C. Singura problema creativa (IA)

Clasificarea cunostintelor

1. Dupa modul de dobandire :

- apriori : cunostinte universal valabile ce nu pot fi negate;

- aposteriori : dobandite in urma unei experiente.

2. Dupa gradul de generalitate :

- semnale : forma primara, contin date utile dar si zgomote;

- date : se obtin din semnal dupa ce au fost curatate de zgomot sau informatii auxiliare;

- informatii : extrase din date prin alocarea specificatiei fizice (unit. de masura, nume, param., valoare);

- cunostinte : informatii specializate;

- metacunostinte : cunostinte despre cunostinte.

3. Despre continut :

- procedurale : cum se ajunge la un anumit rezultat;

- declarative : asignate valorii de adevarat sau fals;

- tacite : dobandite neconditionat.

Logica matematica

Logica matematica elimina ambiguitatile din limbaj pentru a obtine un cadru riguros pentru rationamente.

Enunt – constructie lingvistica ce cuprinde subiecte/subiecte si parte predicativa

Propozitia – enunt cu toate subiecte instatiate.

Predicatul este o propozitie cu cel putin o variabila, adica are cel putin un subiect neintantiat.

Arietate – numarul de subiecte neintantiate dintr-un predicat.

Enunturile compuse sunt compuse din enunturi atomice.

Conective logice : negatie, conjunctia (si), disjunctia (sau), implicatia si echivalenta

Acumularea de cunostinte

•Acumularea de cunostinte: aceasta presupune pe langa achizitia de noi cunostinte si clasificarea lor pentru a fi regasite atunci cand sunt relevante. Procesele de observare a realitatii, invatare, abstractizare si achizitie au fost foarte mult dezvoltate incat au devenit subdomenii SBC. Elementele cu semnificatie poarta denumirea de piese de cunoastere.

Tipuri de piese:

•Concept (variabila predicativa/functionala) - ce condenseaza rezultatele abstractizarii si specifica criteriile de apartenenta a obiectelor la clase.

•Instanta (constanta) – materializata in obiecte individuale

Codificarea cunostintelor

•Codificarea cunostintelor – dezvoltarea de programe care sa codifice si sa proceseze piesele de cunoastere. Cu toate ca poate fi realizata codificarea si prin programare clasica, apar noi necesitati privind gasirea pieselor de cunoastere in baza, efectuarea de rationamente. Ca urmare intre structurile ce reprezinta o piesa de cunoastere sunt necesare legaturi prin care sa fie usor regasite piesele de cunoastere inrudite.

•Cu toate ca etapele de acumulare si codificare par a fi secventiale, dezvoltarea oricarui SBC presupune un process iterativ in care etapele se repeta de mai multe ori.

Model de reprezentare – calcul cu predicate

- variabila predicativa (predicat) : relatie intre doua concepte

- instantta (obiect) : constanta

Categorii gramatiale : predicat, subiect/subiecte.

Etape :

- Acumularea de cunostinte in limbaj natural

- Descompunerea asertiune in asertiune elementara

- Legarea asertiunilor elementare prin conctive logice

- Definire predicate – codificare

Problema consistentei Bazei de Cunostinte

Intr-un sistem de productie in care avem un numar mare de productii se poate obtine ca rezultat o expresie ca fiind adevarata in acelasi timp falsa => baza de cunostinte este incosistenta.

O baza de cunostinte este consistenta daca pornind de la orice vaza de fapte initiala nu se obtine atat o valoare de adevarat pentru o formula, cat si o valoare de fals.

Strategii de control inainte

Strategia de control inainte porneste de la ipoteza si aplica succesiv piesele de cunoastere aplicabile pana la gasirea solutiei sau pana cand multimea candidata devine vida.

Tranzitia de la un context la altul este rezultatul aplicatii unei piese de cunoastere.

Starea finala poate sau nu sa satisfaca obiectivul.

Este o strategie in care favorizam piesele de cunoastere cu mai multe formule in premisa.

Cautare informala (metrica)

De multe ori pot fi stabilite metrice prin care se evalueaza costul sau eficienta atingerii obiectivului.

f(n) = g(n) + h(n)

f(n) – costul drumului de la nodul de start la obiectiv

g(n) – costul drumului parcurs de la nodul de start la nodul n

h(n) – estimarea costului de la nodul n la obiectiv. Poate fi mai mare sau mai mic decat costul real.

Situatii :

- g(n) (h(n) este o necunoscuta) – metoda de cautare optimala

- h(n) (g(n) este o necunoscuta) – genereaza metodele de gradient

- g(n),h(n) – algorimt A/A*

ni fiu al lui n => g(ni) = g(n) + c(n,ni)

c(n,ni) – costul tranzitiei de la n la ni

Metoda de cautare gradient

Open_list={nod_start(0)}

Closed_list=Ø

while (open_list≠Ø) {

n=prim_nod(open_list)

remove(n, open_list)

add(n, closed_list)

if(obiectiv(n)) exit(succes) //testeza potrivire obiectiv

expand(n) // genereaza toate nodurile fiu ni

calculeaza g(ni) // g(ni)=g(n)+c(n,ni)

if (ni ∈ openlist AND g(ni_new) < g(ni_old)) asigneaza {g(ni) = g(ni_new) AND pointer parinte}

if (ni NU∈ openlist) ADD (ni, openlist)

}

exit(esec)

Incosistenta sistemelor de productii

Un sistem de productii este inconsistent daca pornind de la o baza de fapte initiala se poate obtine validarea si invalidarea unei expresii.

Exemplu:

•BF (constructie=rezistenta, cutie=aspectoasa, conectare=puternica)

•Sistemul de productii:

P1: IF (constructie=rezistenta AND cutie=aspectoasa)

THEN (confectie=aluminiu)

P2: IF (cutie=aspectoasa) THEN (surub = cap ingropat)

P3: IF (conectare=puternica)THEN (surub= NOT(cap ingropat))

Castig informational

O metoda de organizare a unei baze de cunostinte este si cea bazata pe castig informational. Porneste de la algoritmul ID3 propus de Rose Quinlian.

Un exemplu este cel de generare a regulilor de clasificare pe baza unui set de exemple.

Daca notam cu t1,t2,t3 teste care dupa executie intorc valori in domenii cunoscute, de ex: (t1a,t1b,t1c), (t2a,t2b,t2c), (t3a,t3b) si cu d1 si d2 clase de defecte, exemplele se scriu:

t1a,t2b,t3a:d1

t1a,t2a,t3c:d2 .....